

Laboratory 2: Finite State Machines and Introduction to FPGA

Google Doc Link:

https://docs.google.com/document/d/1Cw_ePmO_1bHOaKCPdZs2-vhDSCnP42T0/edit?usp=sharing&oid=110924416719706935733&rtpof=true&sd=true

Objectives

1. Understand and be able to write finite state machines in Verilog HDL.
2. Be able to sanitize noisy inputs from mechanical or physical issues.
3. Be able to implement time-division multiplexing.
4. Understand and be able to use Vivado to synthesize, implement, generate bitstream, and program the Basys 3 board.

Part 1: Finite State Machines

Finite state machines (FSM) is an abstraction for sequential circuits with centralized states of operation. In this part, you will learn how to implement finite state machines in Verilog.

Setup

Clone the assignment from this repository:

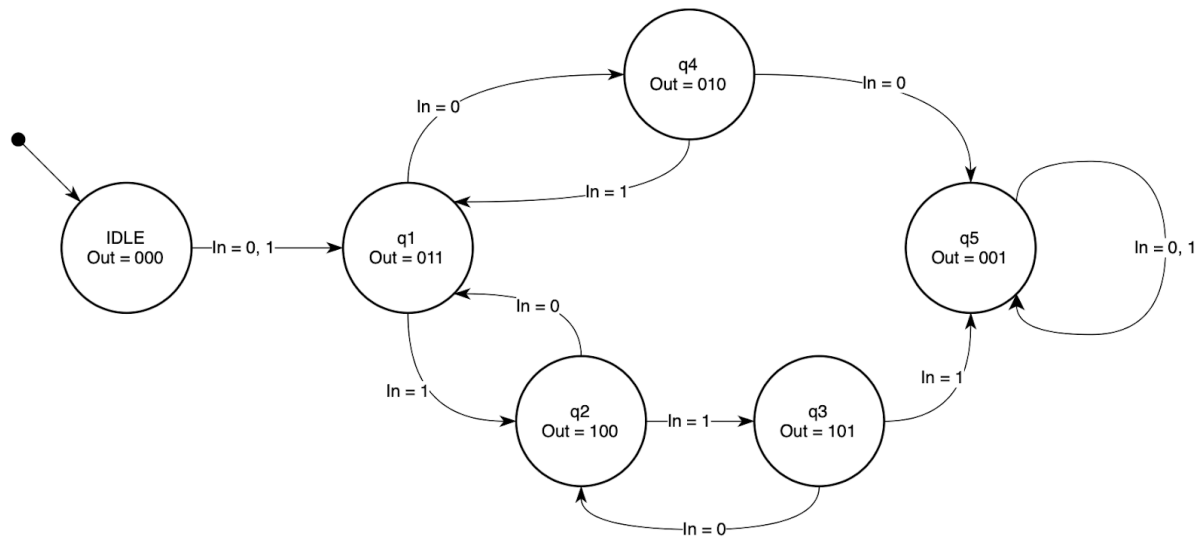
<https://github.com/2110363-HW-SYN-LAB/lab2>

- a. Your task is to edit Verilog files (.v files) in each problem inside “part1” directory. **You may specify *wire* or *reg* to inputs and outputs as you want.**
- b. After editing codes, verify your codes by running testbench inside the “tests” directory of each problem. You can run by activating an environment that installs Cocotb (in case you use venv or Conda) and run the python file (`python <python_file>`).

Problem 1

Implement the **Moore finite state machine (FSM)** described by the state diagram below. The FSM starts in the IDLE state and transitions between states based on a 1-bit input signal. Use a synchronous active-high reset to return the FSM to the IDLE state.

Name the module as `moore_fsm`. The example state diagram illustrates all state transitions and state-dependent outputs.



Inputs

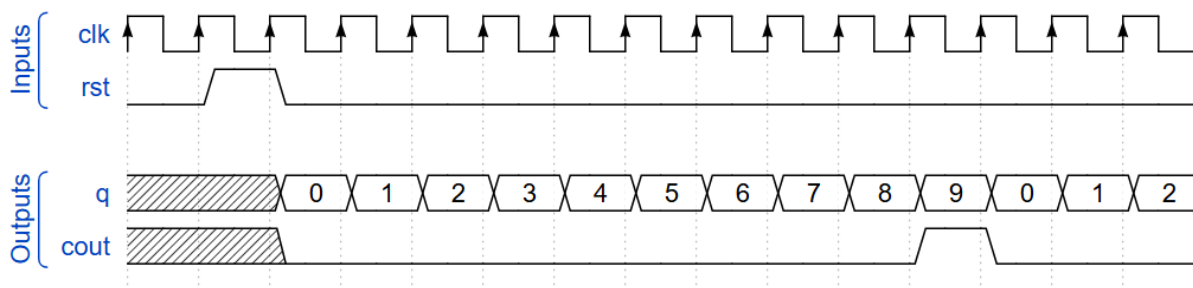
- **clk**: Clock signal that controls the finite state machine
- **rst**: Synchronous reset signal (active high) used to reset every outputs to zero
- **in**: 1-bit input signal that determines state transitions

Outputs

- **out**: 3-bit output signal determined solely by the current state

Problem 2

Implement the BCD counter, which counts the number from 0 to 9 and returns to 0 periodically (inclusive) with synchronous active-high reset. Also, output the pulse indicating when the period ends. Name the module as *bcd_counter*. The example of waveform is shown below.



Inputs

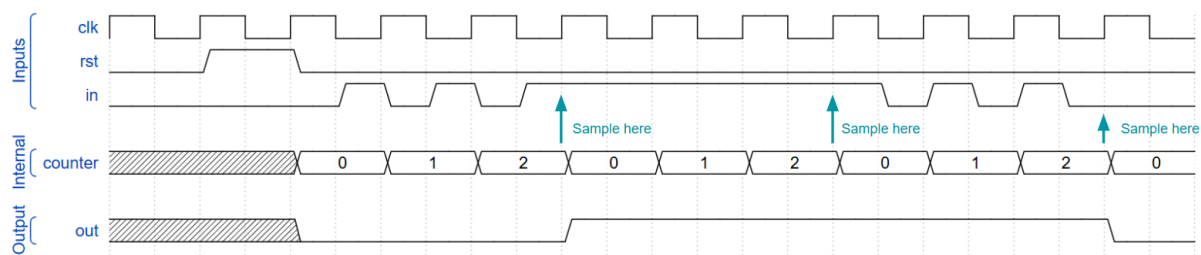
- **clk**: Clock signal that controls the counter to increment its state
- **rst**: Synchronous reset signal (active high) used to reset every outputs to zero

Outputs

- **q**: 4-bit output representing current counting value
- **cout**: 1-bit output representing pulse when the period ends

Problem 3

In the lecture slide, we have shown the concept of debouncing signals by sampling the signals. Implement *debouncer* parameterized module, which samples the input signal every **SAMPLING_RATE** clock cycle. The module also contains synchronous active-high reset, which will reset every output and internal counter to zero. The example of waveform when **SAMPLING_RATE** is 3 is shown below.



Inputs

- **clk**: Clock signal that controls the debouncer
- **rst**: Synchronous reset signal (active high) used to reset every outputs and internal counter to zero
- **data_in**: Input signal that have to be debounced

Outputs

- **data_out**: Debounced output signal

Parameters

- **SAMPLING_RATE**: The sampling rate of the debouncer. Set the default value of this parameter to 3 clock cycles.
- **COUNTER_WIDTH**: The number of bits of reg representing a counter. For example, if this parameter is 2, the counter is 2-bit wide (can count from 0 to 3). Set the default value of this parameter to 2.

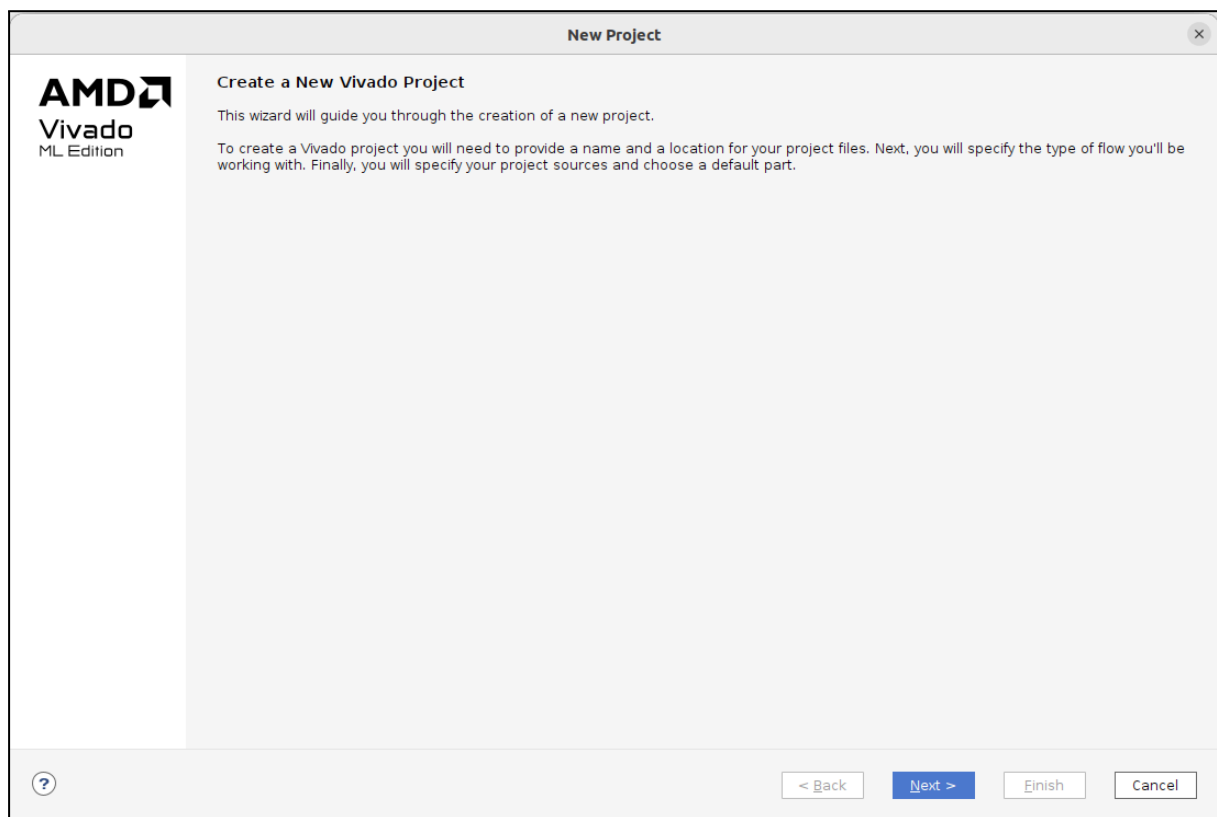
**** Checkpoint: Call TA to check your work ****

Part 2: Introduction to Vivado

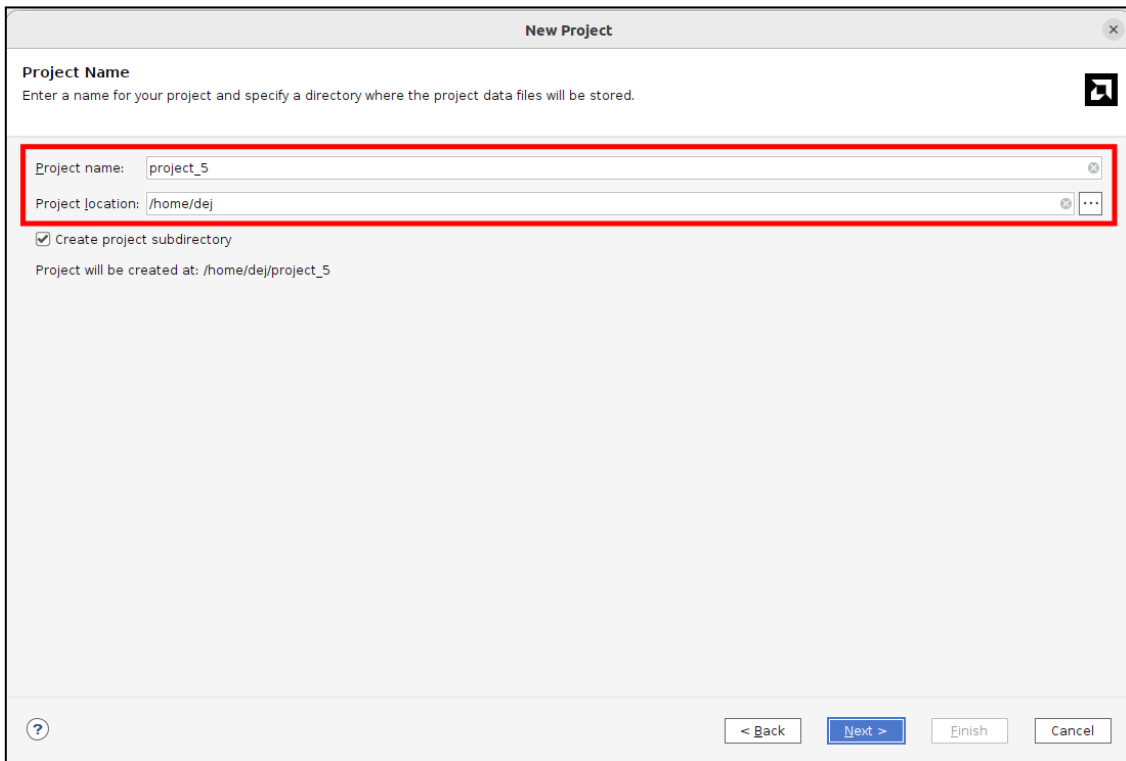
Vivado is a tool used to convert your hardware description language (in this case, Verilog) into a bitstream to program your fpga board. In this part, you will play with a 7-segment display as briefed by the lecturer. Please follow the steps below.

Step 1: Creating Vivado Project

- A. Open Vivado Program
- B. On the tools bar on the upper left
 - a. Select File -> Project -> New
 - b. You will see this page, then click Next



C. Enter your Project Name and Project Location Then click Next



New Project

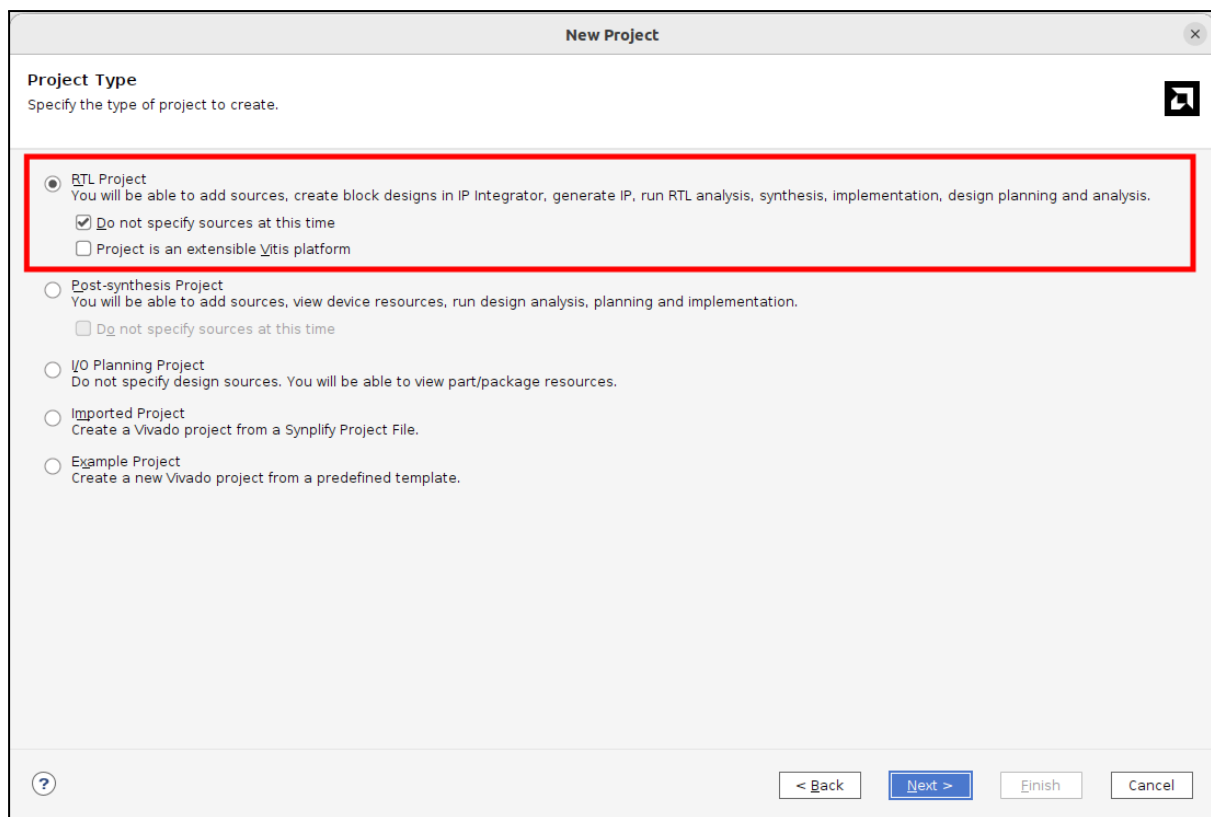
Project Name
Enter a name for your project and specify a directory where the project data files will be stored.

Project name:

Project location:

☒ Create project subdirectory
Project will be created at: /home/dej/project_5

D. Select RTL Project, Then click Next



New Project

Project Type
Specify the type of project to create.

☒ **RTL Project**
You will be able to add sources, create block designs in IP Integrator, generate IP, run RTL analysis, synthesis, implementation, design planning and analysis.
☒ Do not specify sources at this time
☐ Project is an extensible Vitis platform

☐ **Post-synthesis Project**
You will be able to add sources, view device resources, run design analysis, planning and implementation.
☐ Do not specify sources at this time

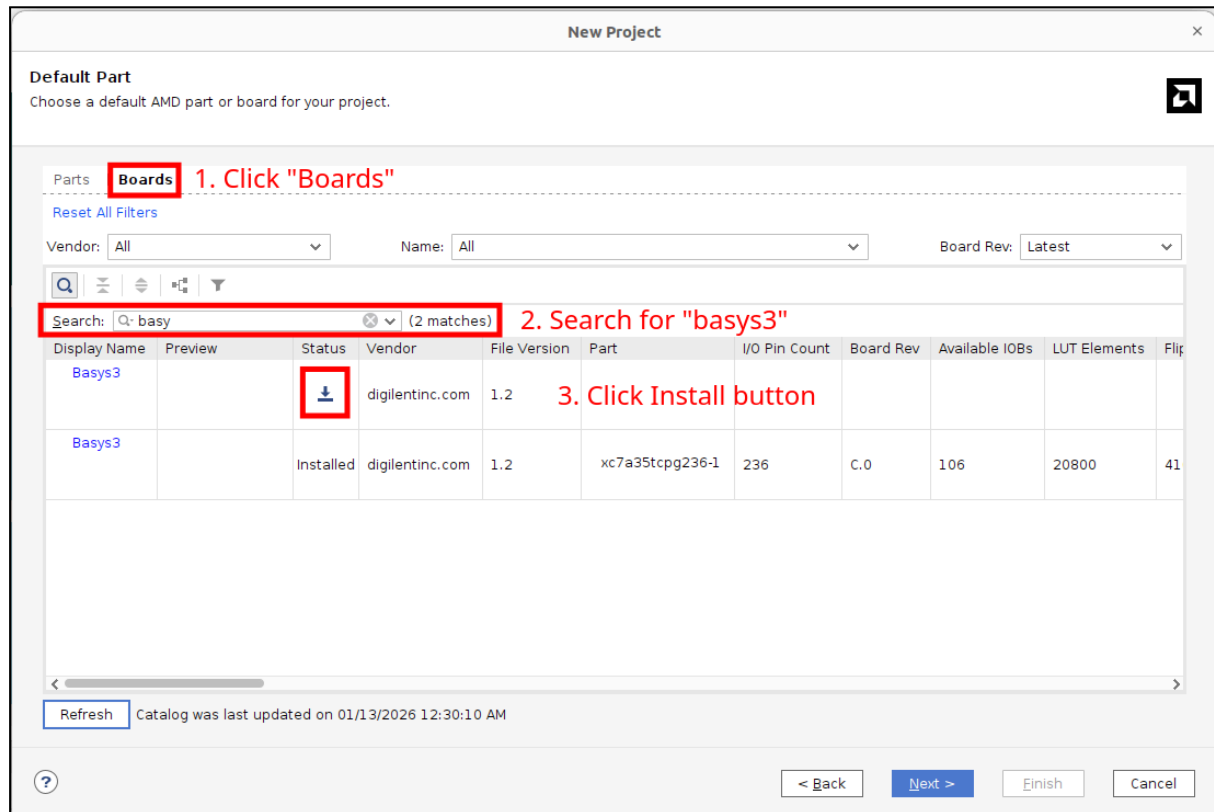
☐ **I/O Planning Project**
Do not specify design sources. You will be able to view part/package resources.

☐ **Imported Project**
Create a Vivado project from a Synplify Project File.

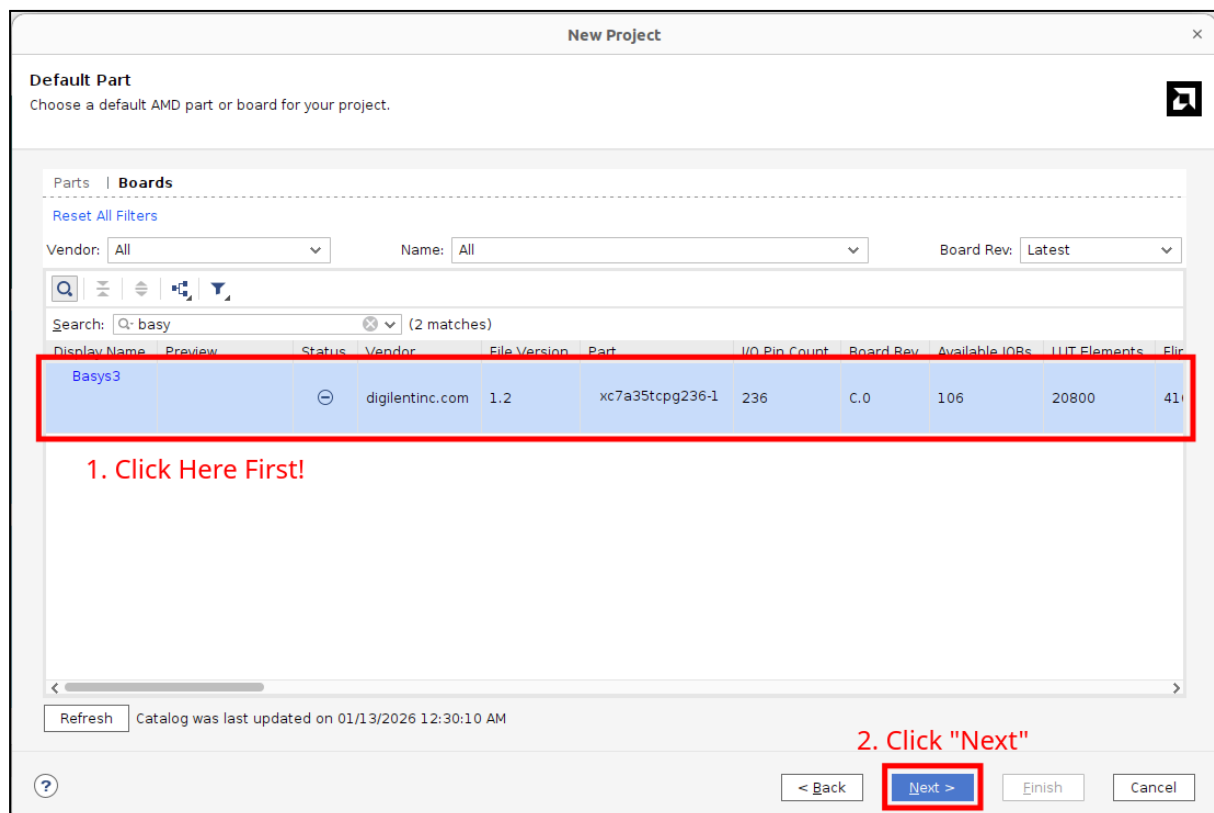
☐ **Example Project**
Create a new Vivado project from a predefined template.

E. Install Basys3 Board File.

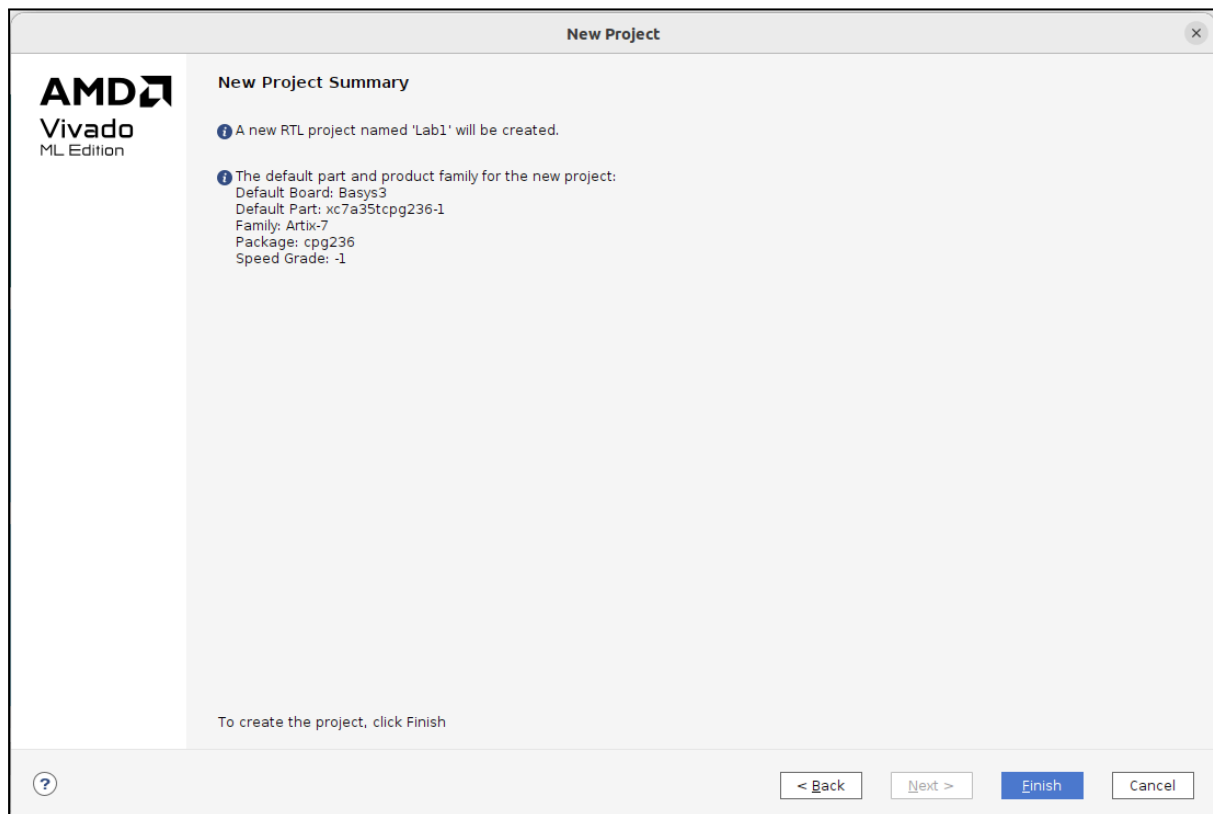
** If Basys 3 is not found, press Refresh button at the bottom left of the panel.



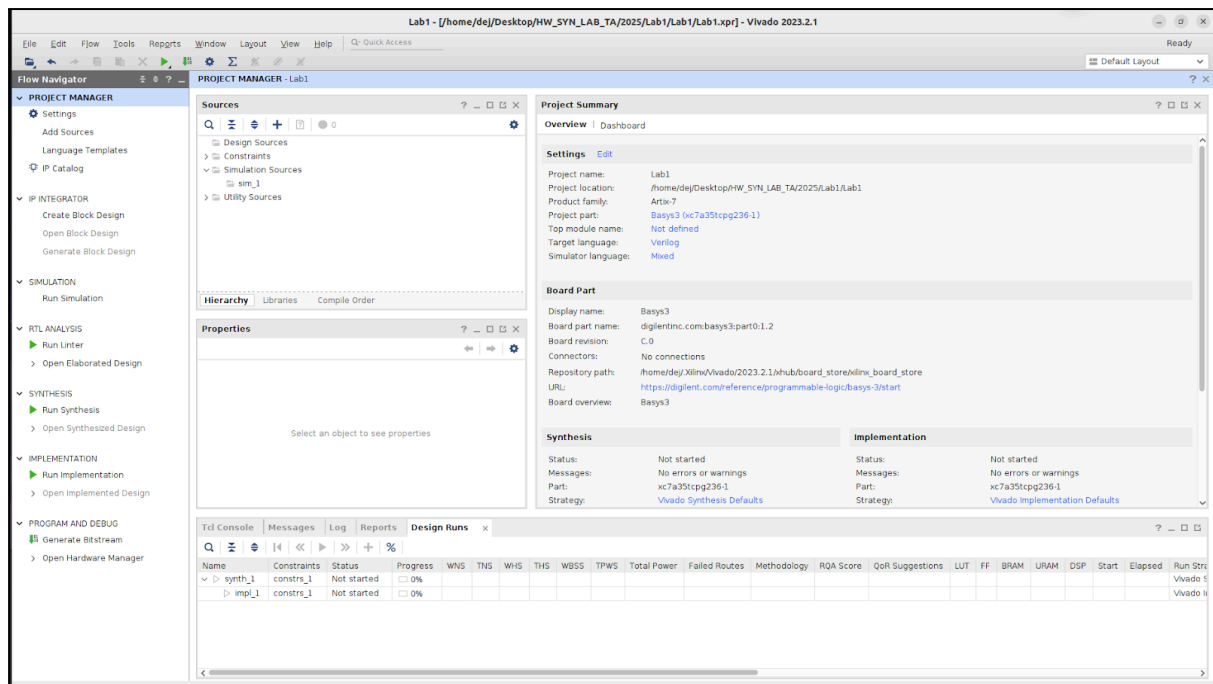
F. Select Basys3 board then click Next



G. Click Finish



H. Now you have created your Vivado project, you should get something like this.



Step 2: Constructing 7-Segment Display Controller

Project Overview

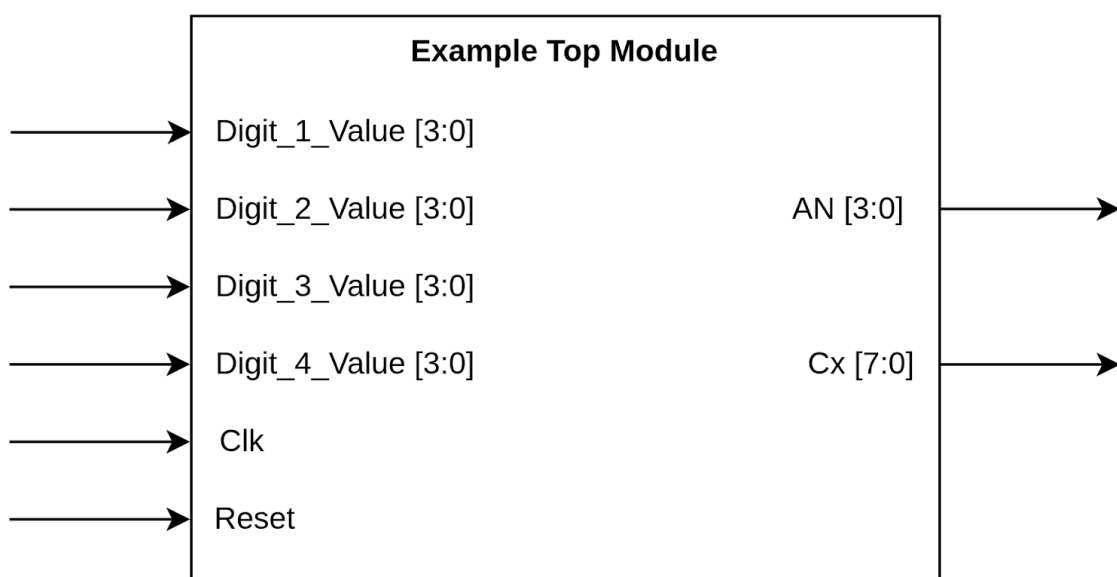
In this project, you will design a Verilog module for the **Basys3 FPGA board** that controls **all four digits** of the on-board **7-segment display** using the board's **switches** and a **button**.

Functional Requirements

- **Input:** A 16-bit value from the onboard switches, where each group of 4 bits represents one 7-segment display digit.
- Each digit is displayed correctly on the 7-segment display, with values ranging from **0 to F in hexadecimal**. The format of the 7-segment for each digit/character must follow the image below.



- The module includes an **active-high, synchronous reset**. When asserted, it resets all internal registers to their initial values.
- The module operates on the **positive edge of the clock**.
- The example of ports of the top-level module (the main module) are shown below.



Inputs

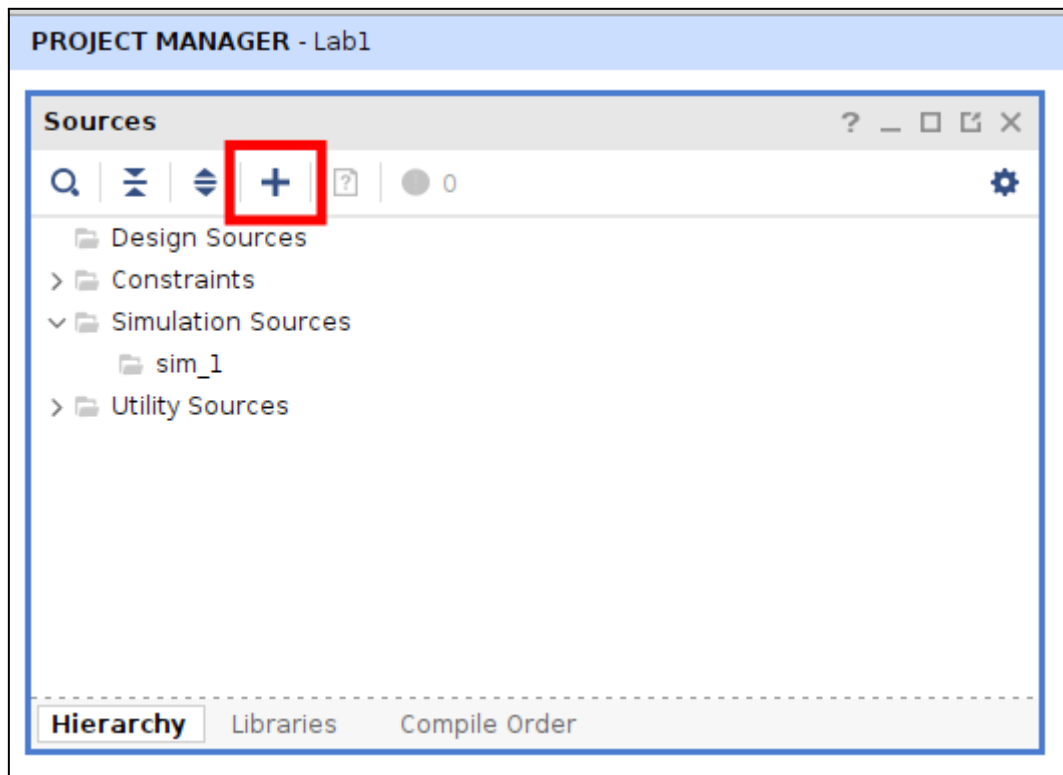
- **Digit_1_Value**: 4-bit number for 1st digit (from the left) in 7-segment display
- **Digit_2_Value**: 4-bit number for 2nd digit (from the left) in 7-segment display
- **Digit_3_Value**: 4-bit number for 3rd digit (from the left) in 7-segment display
- **Digit_4_Value**: 4-bit number for 4th digit (from the left) in 7-segment display
- **Clk**: Clock signal that controls the counter to increment its state
- **Reset**: Synchronous reset signal (active high) used to reset the system

Outputs

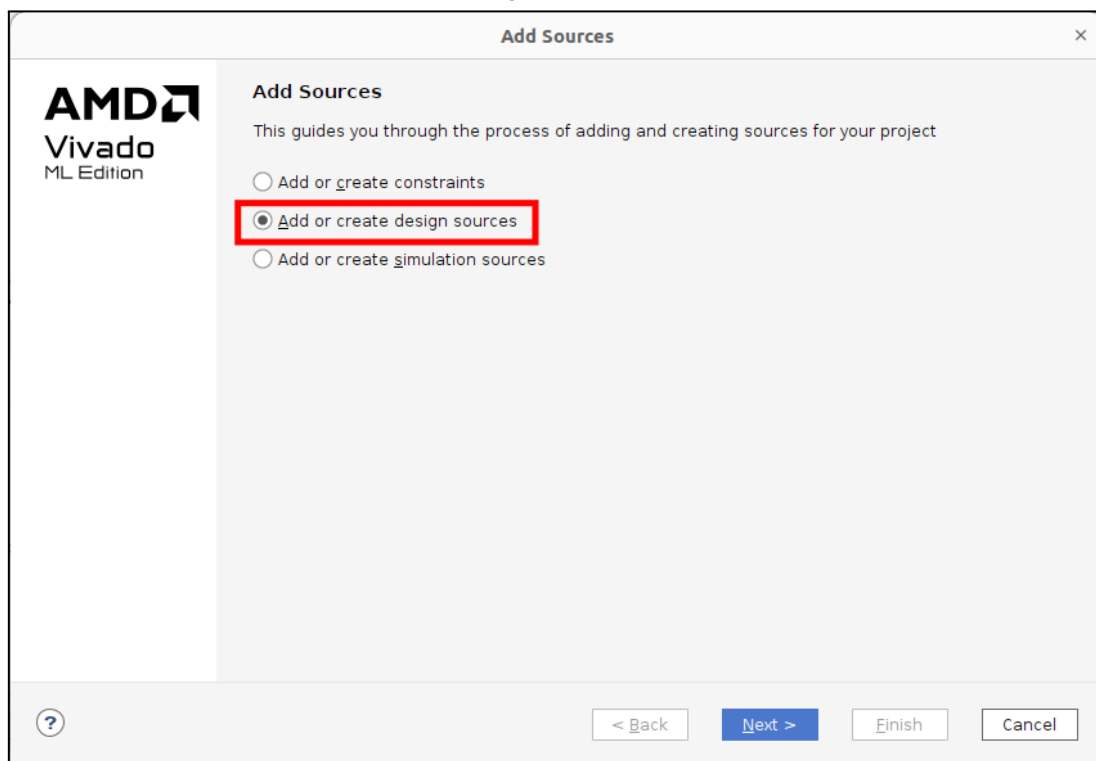
- **AN**: 4-bit output for indicating which digit is active
- **Cx**: 8-bit output for indicating which segments of digit are active, **including decimal point**. You can design which bit refers to which segment as you want. For example, you can design LSB of this output to refer to a decimal point of 7-segment display.

Your Task

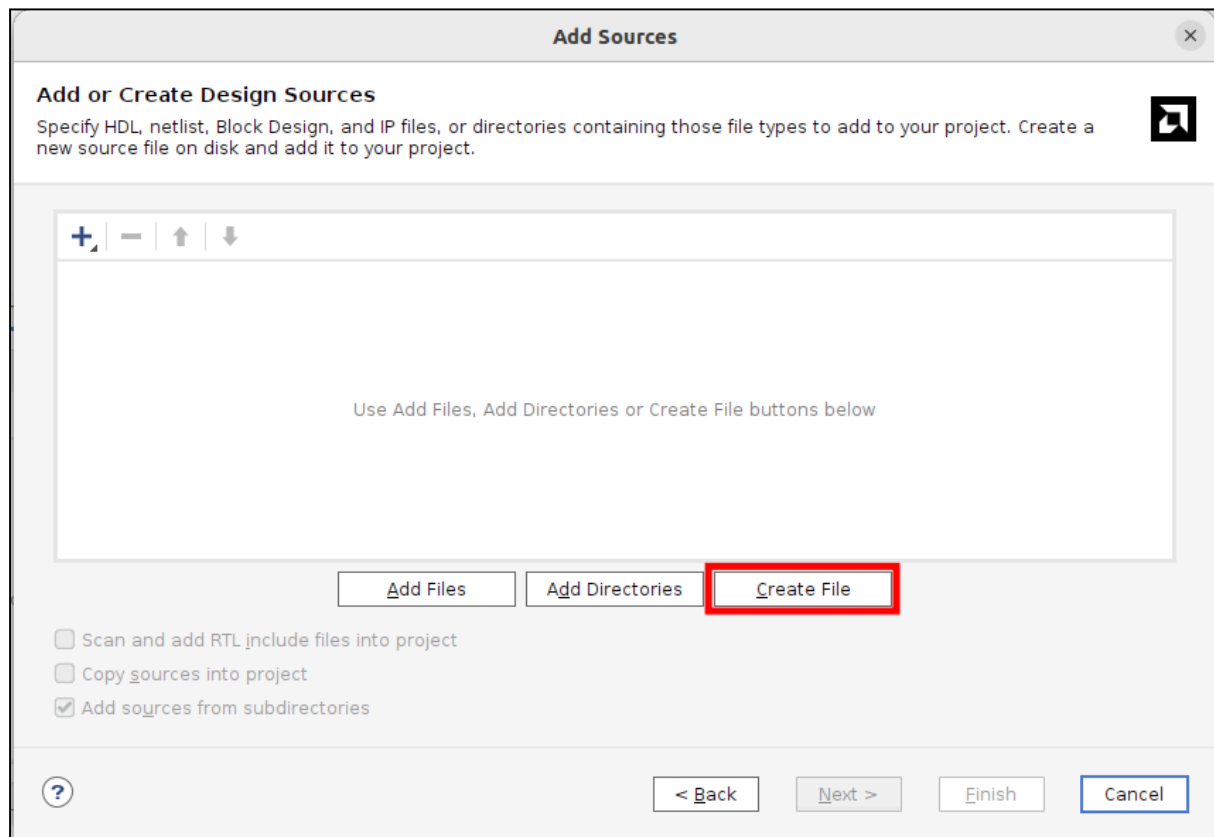
- A. Create a new Verilog file
 - a. Click the “+” sign on the source window



- b. Select “Add or create design sources”

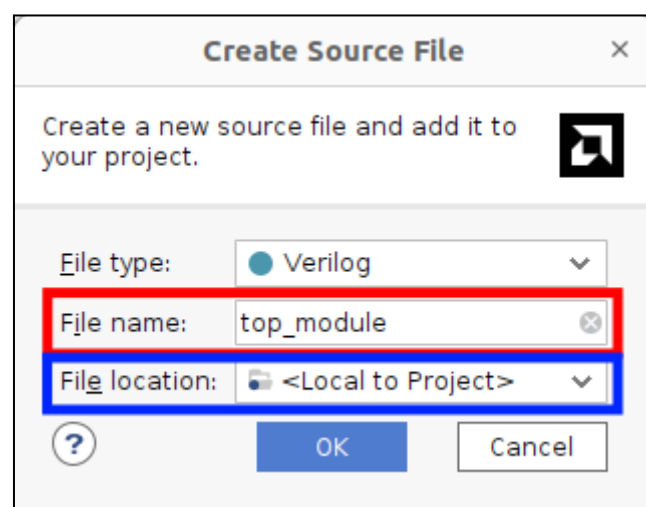


c. Click “Create File”

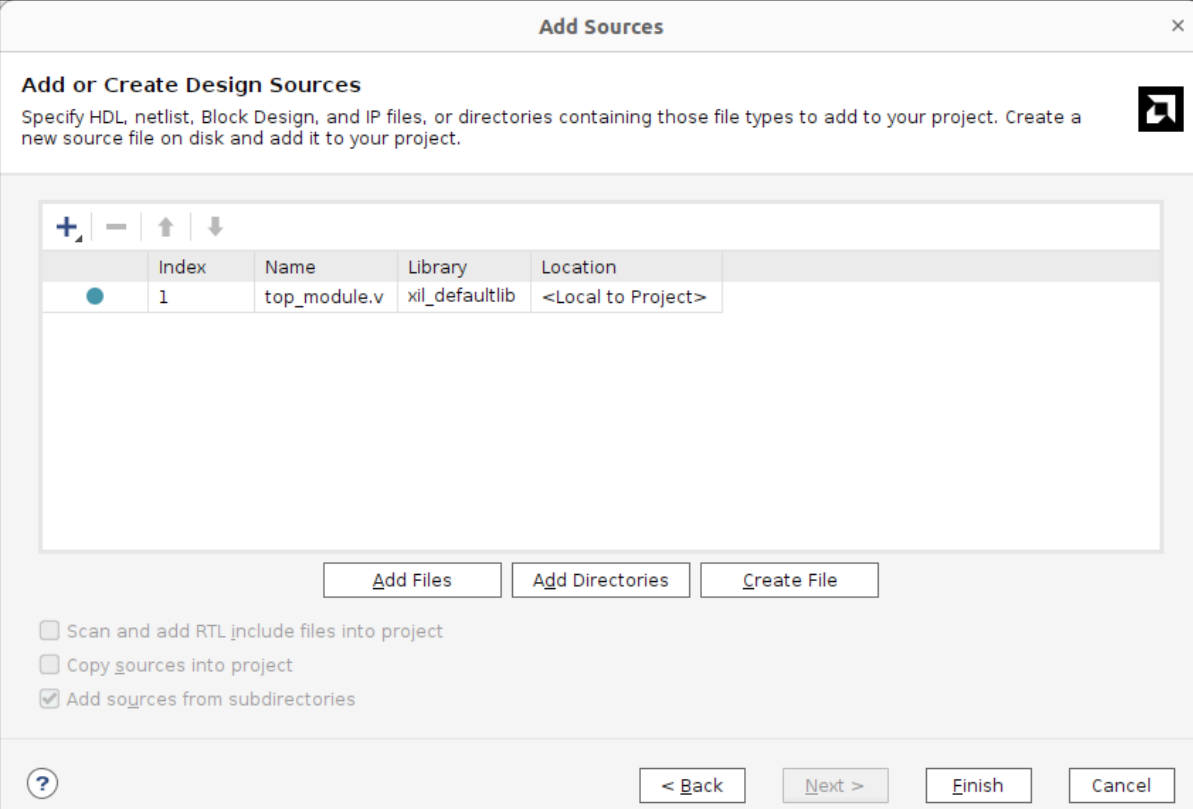


d. Create a new file!

- i. Fill in the file name
- ii. Fill in the File location if wanted (default location is <your project path>/<project name>.srcs/sources_1/new/)
- iii. Click Ok



e. New file is added, click finished



Add Sources

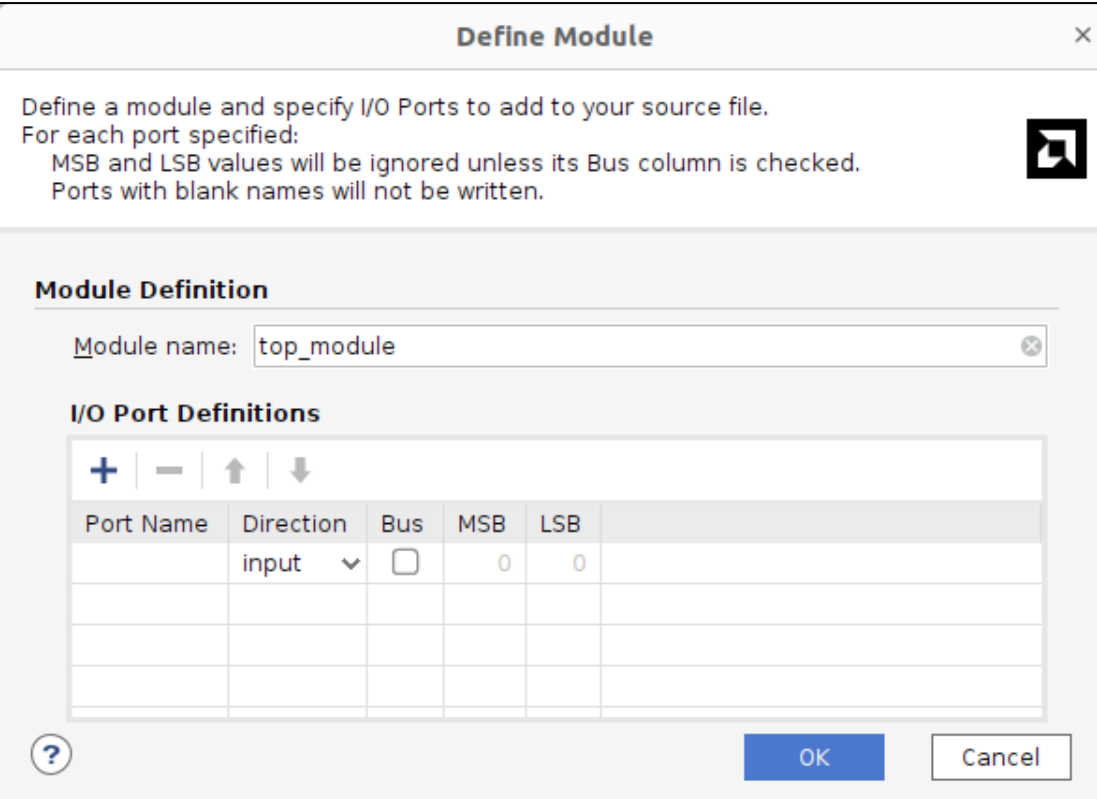
Add or Create Design Sources

Specify HDL, netlist, Block Design, and IP files, or directories containing those file types to add to your project. Create a new source file on disk and add it to your project.

	Index	Name	Library	Location
●	1	top_module.v	xil_defaultlib	<Local to Project>

☐ Scan and add RTL include files into project
☐ Copy sources into project
☒ Add sources from subdirectories

f. Click OK



Define Module

Define a module and specify I/O Ports to add to your source file.
For each port specified:
MSB and LSB values will be ignored unless its Bus column is checked.
Ports with blank names will not be written.

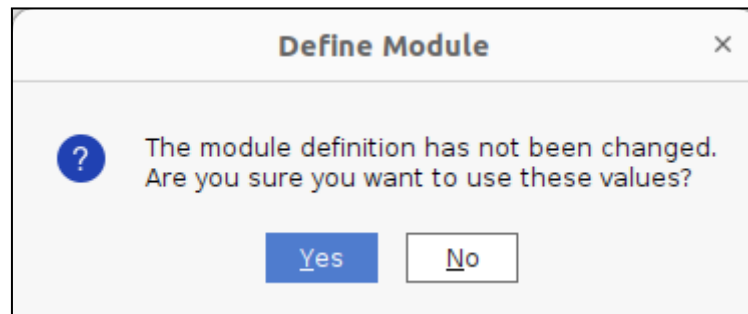
Module Definition

Module name:

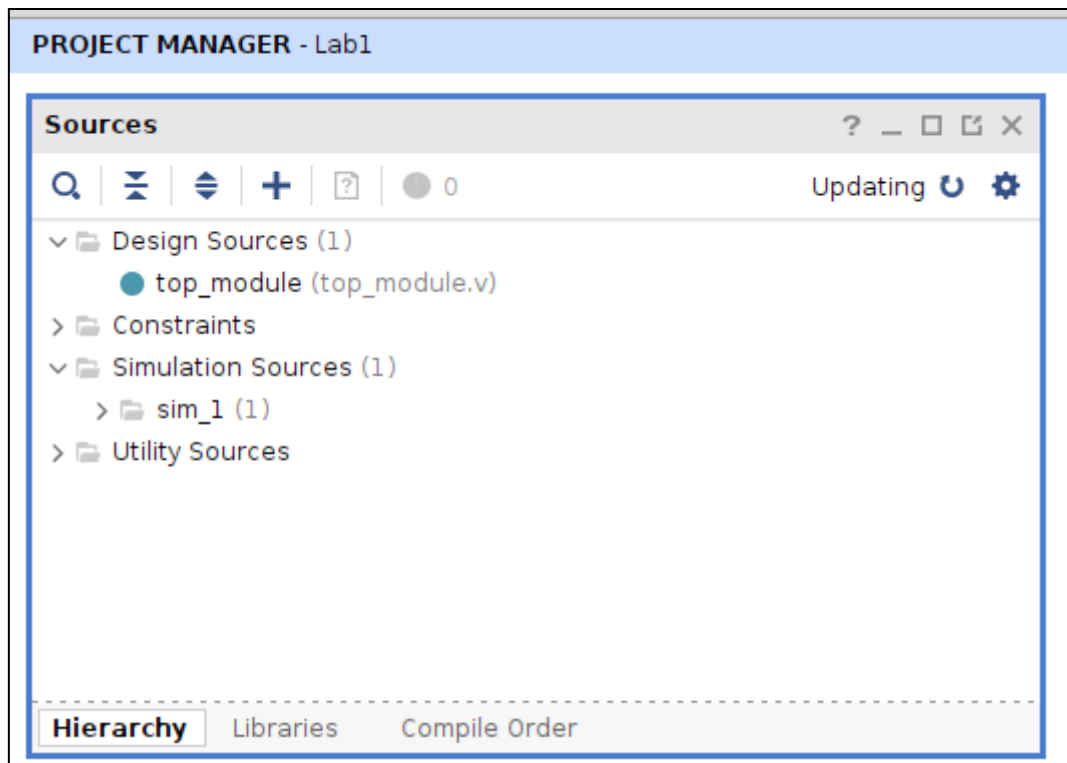
I/O Port Definitions

Port Name	Direction	Bus	MSB	LSB
	input	☐	0	0

g. Click Yes



h. Your new module is added!



B. Modify the created file to match functional requirements.

- You can create more files if needed.
- Design should be modular.

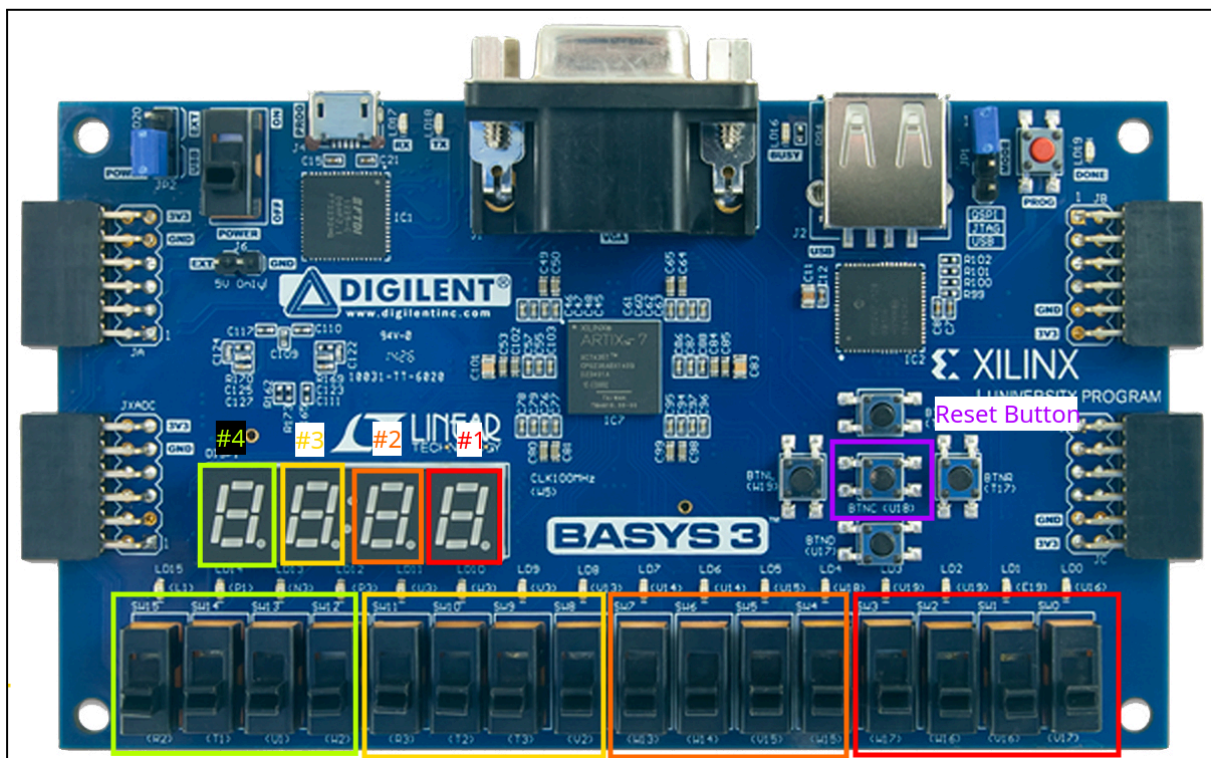
C. Create a Testbench to test your design (Optional)

Step 3: Creating Constraint File

A constraint file is used to map the I/O port of your top module file onto the real hardware!

I/O Specifications

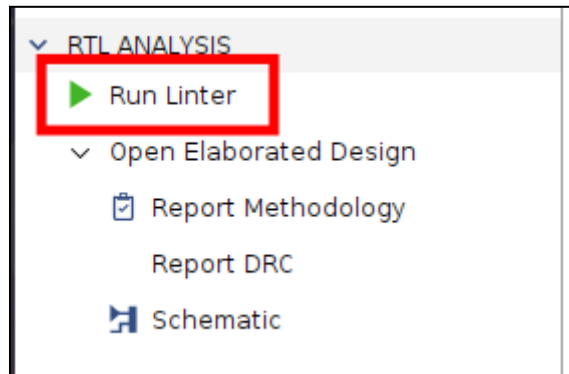
- SW0-3 is the value of the #1 digit of the 7-Segments display where SW3 is the MSB and SW0 is the LSB
- SW4-7 is the value of the #2 digit of the 7-Segments display where SW7 is the MSB and SW4 is the LSB
- SW8-11 is the value of the #3 digit of the 7-Segments display where SW11 is the MSB and SW8 is the LSB
- SW12-15 is the value of the #4 digit of the 7-Segments display where SW15 is the MSB and SW12 is the LSB
- BTNC is used as the reset button
- Clock is connected to the 100MHz Oscillator on Basys 3 board



Your Task

A. Run Linter

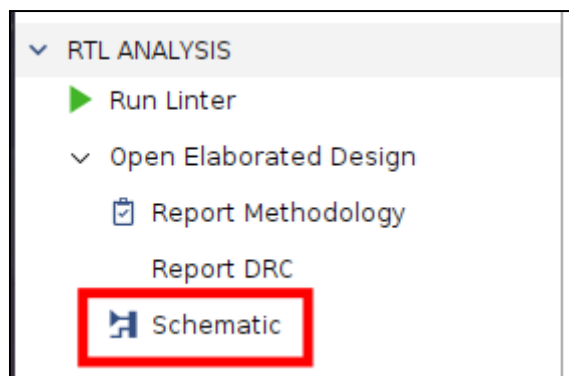
- a. On the left side in the flow navigator tab, Click run Linter



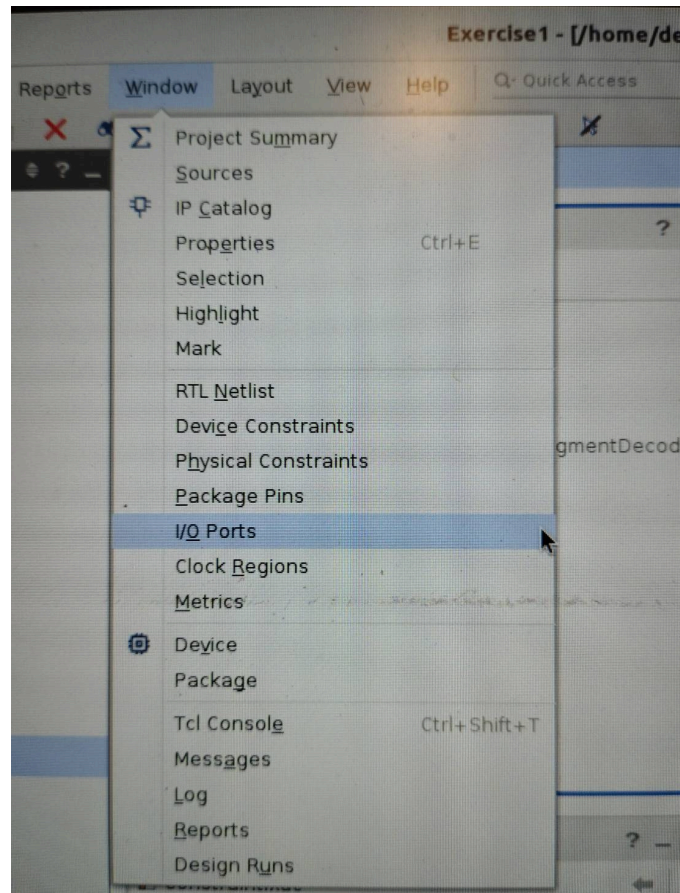
- b. Wait for it to complete

B. Open the schematic

- a. Click Schematic



- C. Open the I/O port window
- On the Top panel select Window -> I/O Ports



- This window should show up

 A screenshot of the 'I/O Ports' window in the Xilinx IDE. It displays a table with columns: Name, Direction, Board Part Pin, Board Part Interface, Neg Diff Pair, Package Pin, Fixed, Bank, I/O Std, Vcco, Vref, Drive Strength, Slew Type, Pull Type, and Off-C. The table lists 22 ports, including AN (4), Segments (8), SW (8), and Scalar ports (2).

Name	Direction	Board Part Pin	Board Part Interface	Neg Diff Pair	Package Pin	Fixed	Bank	I/O Std	Vcco	Vref	Drive Strength	Slew Type	Pull Type	Off-C
AN (4)	OUT					✓	34	LVC MOS33*	3.300		12		NONE	FP_V
Segments (8)	OUT					✓	34	LVC MOS33*	3.300		12		NONE	FP_V
SW (8)	IN					✓	14	LVC MOS33*	3.300				NONE	NONE
Scalar ports (2)														

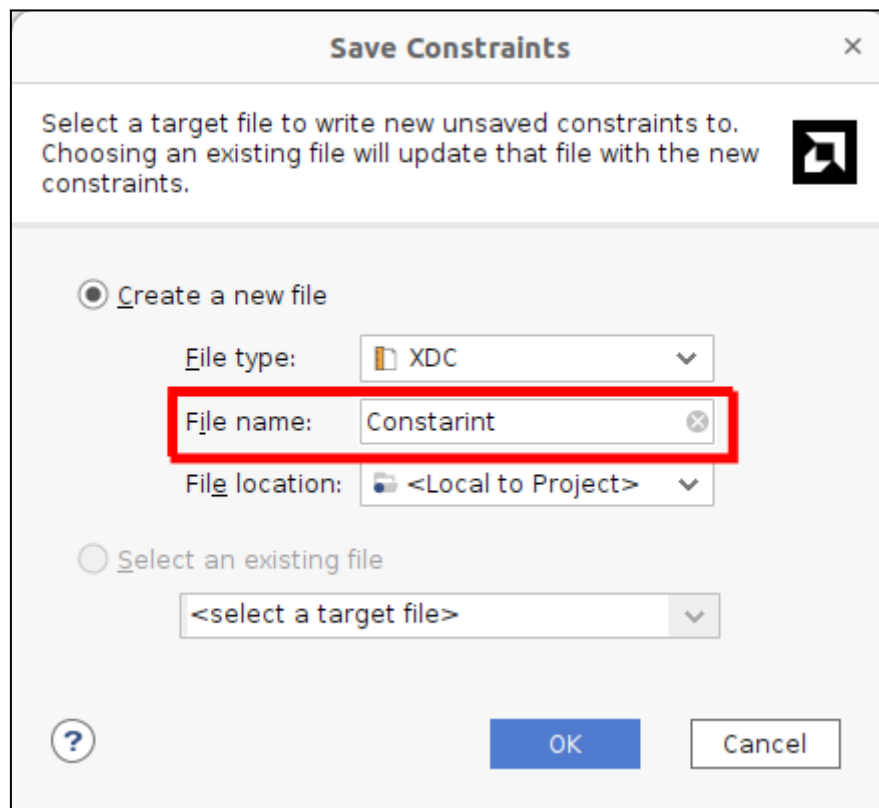
- D. Assign the pin package and I/O standard for each port (The I/O standard should be LVC MOS33).

 A screenshot of the 'I/O Ports' window with two red boxes highlighting specific configurations. The first box highlights the 'Package Pin' column, showing 'W4' for AN[3]. The second box highlights the 'I/O Std' column, showing 'LVC MOS33*' for AN[3] and 'default (LVC MOS18)' for other ports.

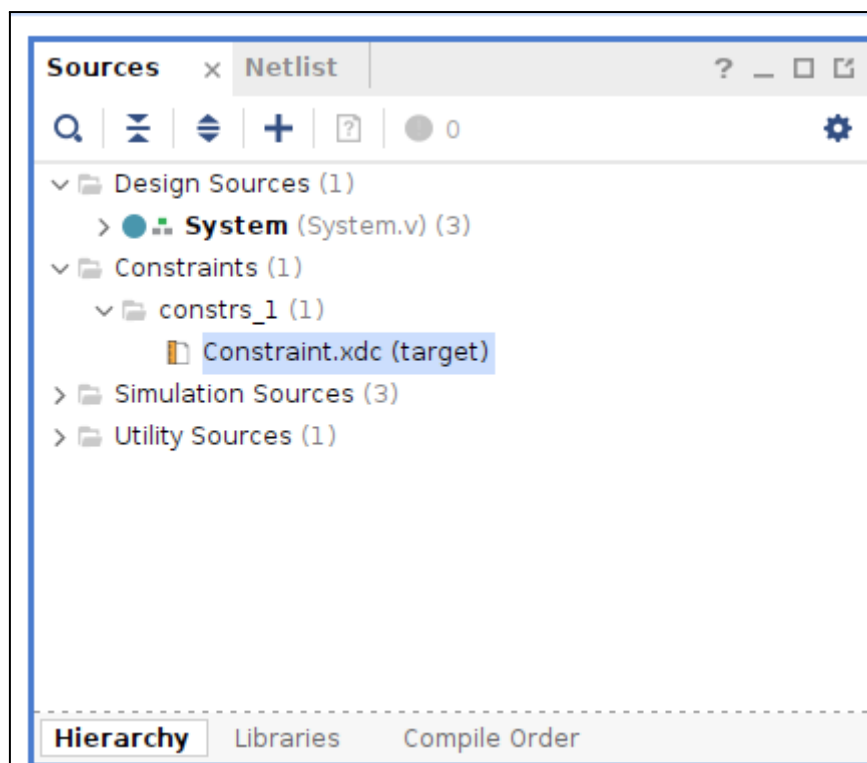
Name	Direction	Board Part Pin	Board Part Interface	Neg Diff Pair	Package Pin	Fixed	Bank	I/O Std	Vcco
AN (4)	OUT					☐	(Multi	(Multiple)*	(Multi
AN[3]	OUT				W4	☒	34	LVC MOS33*	3.300
AN[2]	OUT					☐		default (LVC MOS18)	1.800
AN[1]	OUT					☐		default (LVC MOS18)	1.800
AN[0]	OUT					☐		default (LVC MOS18)	1.800
Segments (8)	OUT					☐		default (LVC MOS18)	1.800
Segments[7]	OUT					☐		default (LVC MOS18)	1.800
Segments[6]	OUT					☐		default (LVC MOS18)	1.800
Segments[5]	OUT					☐		default (LVC MOS18)	1.800

- E. Save the constraint by pressing Ctrl + S.

F. Enter the file name then click OK.



G. The constraint file should be generated now.

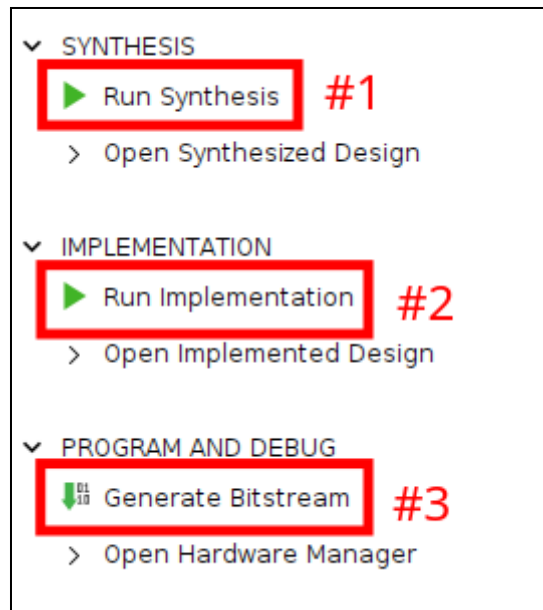


H. Add this line to your constraint file to create a constraint on your 100MHz clock.

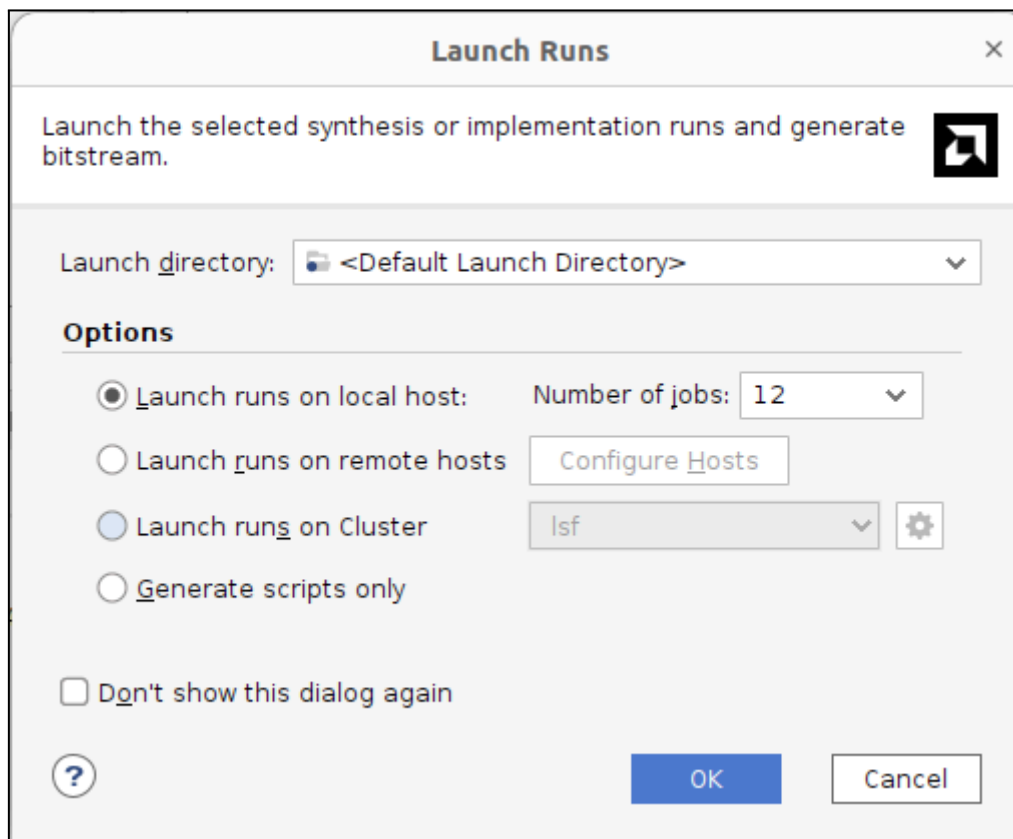
```
create_clock -period 10.000 -name Clk -waveform {0.000 5.000} [get_ports <your  
clock port name on the top module>]
```

Step 4: Running Synthesis, Implementation, and Generating Bitstream

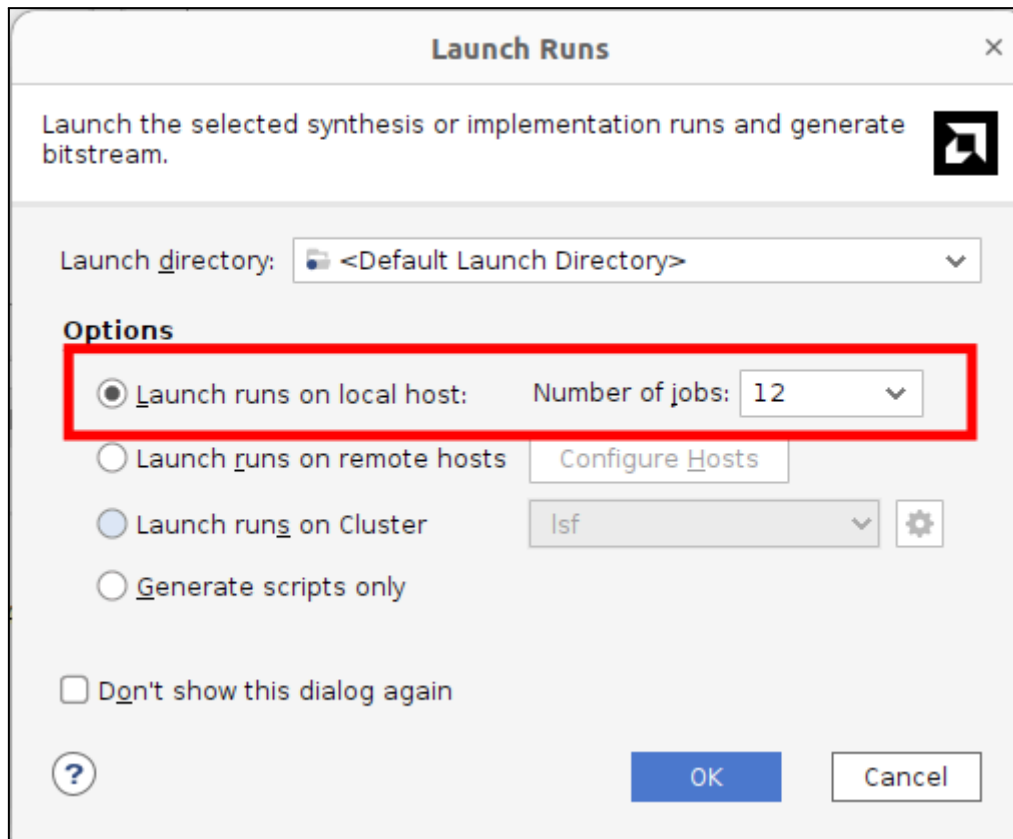
- A. On the flow navigation tab, you can either
- Click Synthesis, Implementation, Generate Bitstream in order
 - Click Run Generate Bitstream (it will do the above step for you)



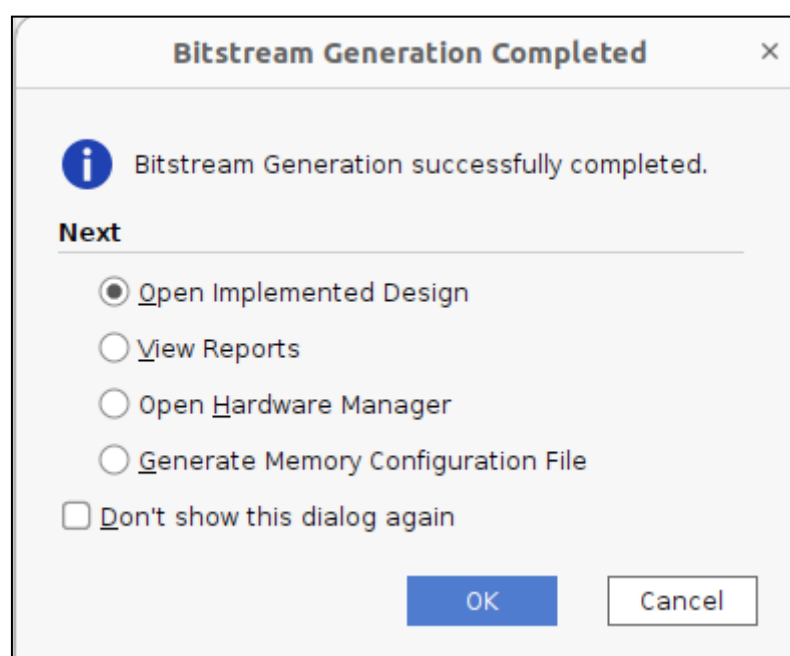
- B. After Clicking
- it will prompt this window to u



- b. Set option to Launch runs on local hosts and a number of job according to your computer resource (more job = faster but require lot of resource + note to never set a jobs to max (it can make vivado terminate itself))



- c. Wait for it to done, it should pop up this window

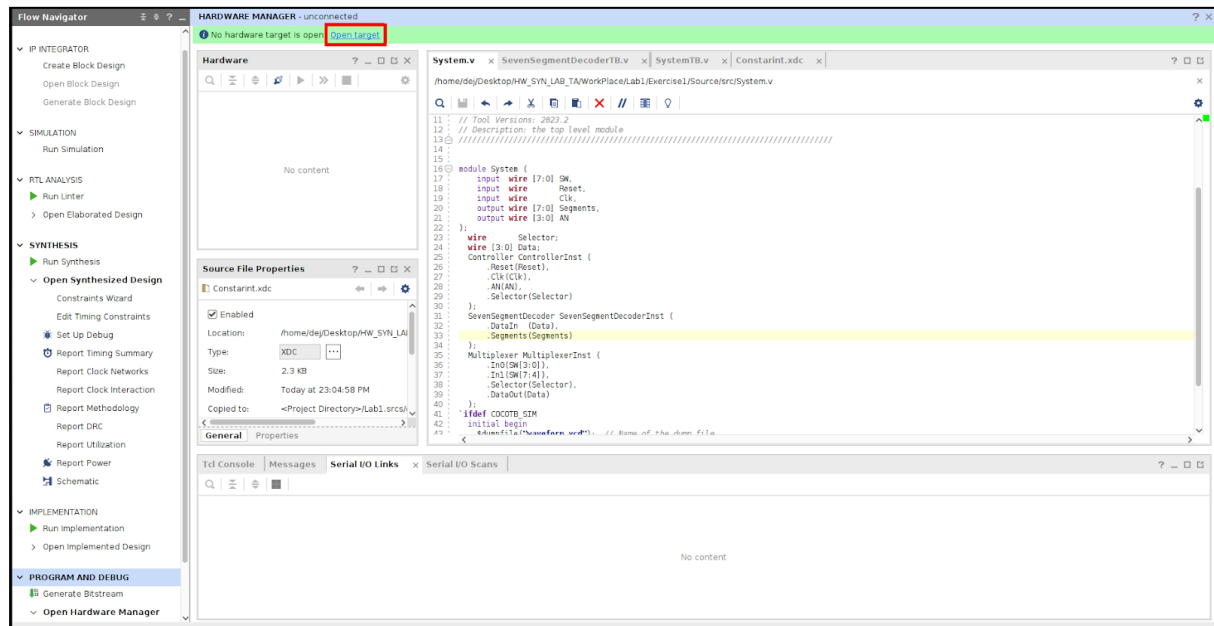


Step 5: Programming Basys3 Board

A. On the Flow navigator, select “Open Hardware Manager”.

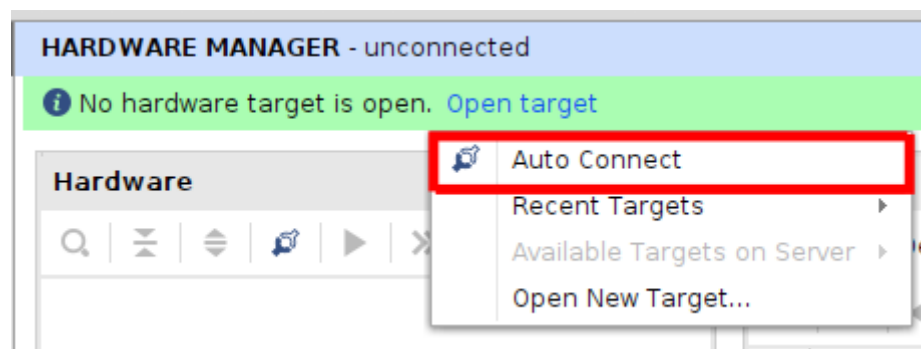


B. Select “Open target”

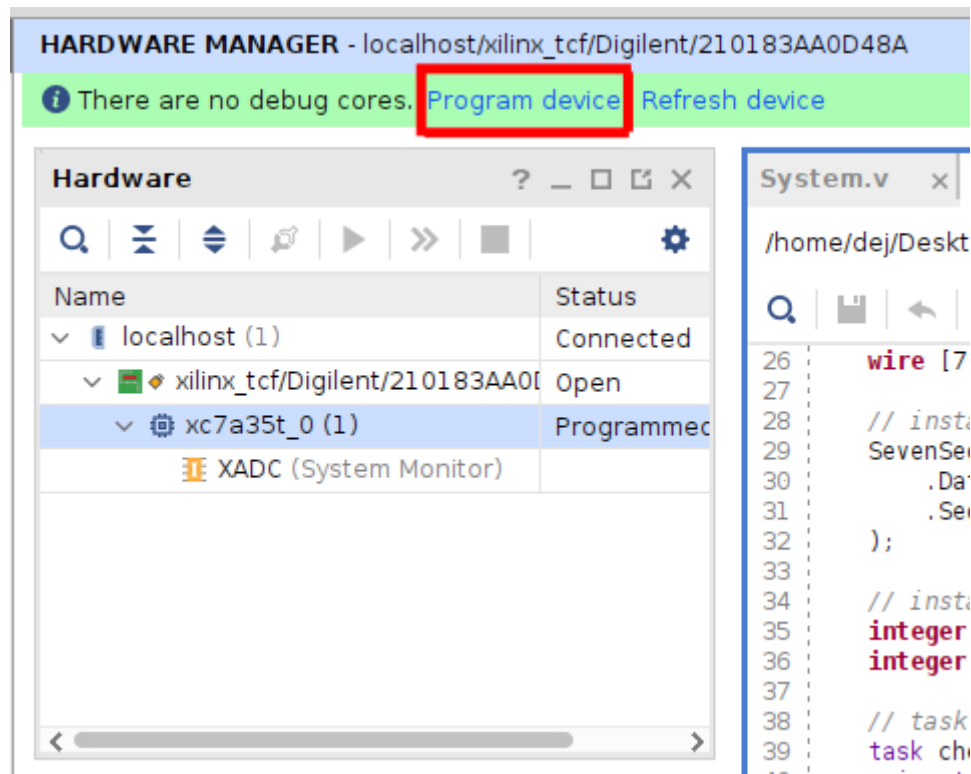


C. Connect your board with your PC/laptop

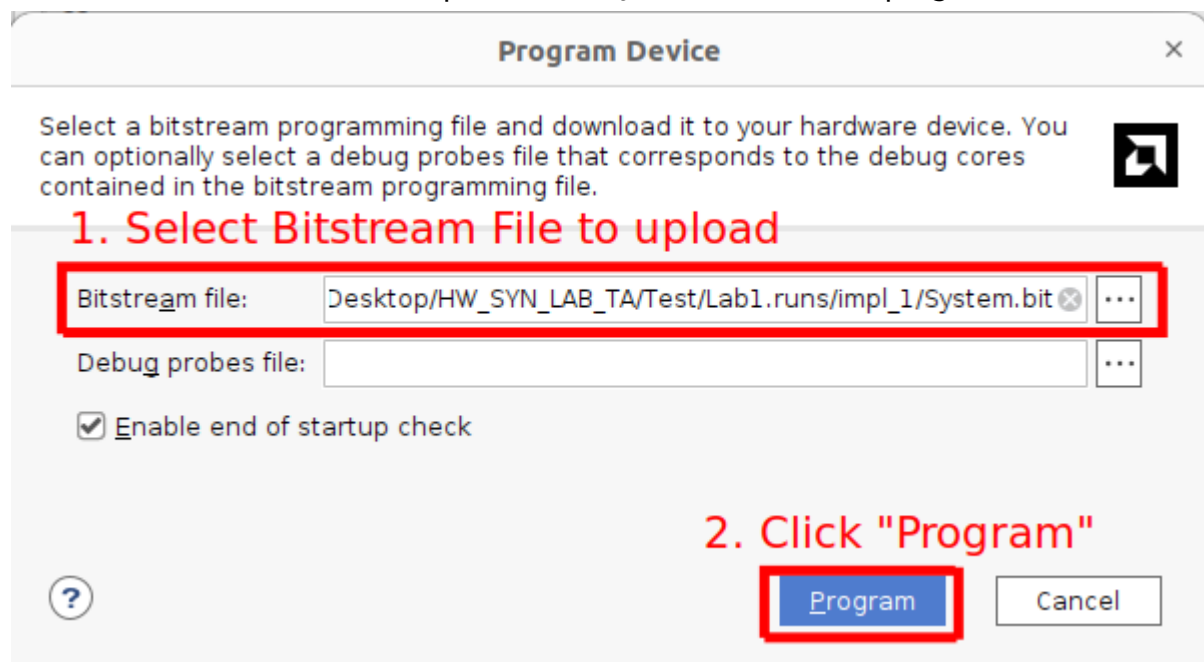
D. Then click “Auto Connect”



E. Then click “Program device”



F. Select bitstream file to upload (usually at xxx) then click program



**** Checkpoint: Call TA to check if your board works as expected as functional requirements, zip all files, and submit in MCV ****